

MANUAL INTEGRAL

Bitácora de Conductores

Registro móvil de conductores, vehículo, rutas,
combustible e incidentes

Guía de usuario · Instalación · Arquitectura · Mantenimiento

Versión del documento: 1.0 · Agosto 2026

Proyecto documentado: Prototipo Xamarin.Forms 5

Autor y responsable del proyecto: Jimmy Toapanta

Fecha de revisión: 13/08/2026

Control del documento

Campo	Detalle
Proyecto	Bitácora de Conductores
Alcance	Manual de usuario y referencia técnica en un único documento
Versión documentada	Prototipo Xamarin.Forms 5
Rama de referencia	master
Estado	Prototipo funcional parcial; requiere modernización y cierre de brechas
Público previsto	Conductores de prueba, evaluadores móviles y mantenedores .NET
Responsable	Jimmy Toapanta
Última revisión	13/08/2026

Cómo usar este manual

La primera mitad acompaña a quien instala y utiliza la solución. La segunda explica cómo está construida, dónde guarda información y qué revisar antes de modificarla o desplegarla.

Este documento distingue lo que funciona hoy de las mejoras previstas. Cuando una integración depende de terceros, la guía lo indica y propone una verificación antes de operar con datos reales.

Contenido

Control del documento	2
Cómo usar este manual	2
Contenido	3
El proyecto en pocas palabras	6
Propósito y alcance	6
Qué resuelve	6
Estado actual	6
Alcance de esta entrega	6
Fuera de alcance	6
Preparación y puesta en marcha	7
Requisitos previos	7
Instalación o incorporación	7
Primera ejecución	7
Operación paso a paso	8
Ingresar al prototipo	8
Administrar un conductor	8
Registrar una recarga	8
Registrar actividad o ruta	9
Registrar un incidente con fotografía	9
Lista de control operativa	9
Problemas frecuentes y recuperación	10
Uso responsable, privacidad y seguridad	10

Preguntas habituales	10
¿Está lista para producción?	10
¿Funciona en iOS?	10
¿Por qué migrar a MAUI?	10
Arquitectura y componentes	11
Vista de arquitectura	11
Responsabilidad de los componentes	11
Configuración, datos e integraciones	12
Parámetros importantes	12
Persistencia y archivos generados	12
Integraciones externas	12
Flujos internos y decisiones de diseño	13
Persistencia de un formulario	13
Captura de incidente	13
Criterios de diseño	13
Desarrollo, pruebas y mantenimiento	14
Entorno de desarrollo	14
Estrategia de pruebas	14
Registro, diagnóstico y soporte	14
Mantenimiento recomendado	14
Despliegue y continuidad	15
Ruta de despliegue	15
Respaldo y recuperación	15

Criterios de aceptación	15
Límites conocidos y evolución	16
Límites de la versión documentada	16
Hoja de ruta sugerida	16
Versiones y ramas	16
Entrega a otra persona	17
Paquete de traspaso	17
Glosario breve	17
Cierre	17

01 · CONTEXTO

El proyecto en pocas palabras

Bitácora de Conductores es una aplicación Xamarin.Forms que explora el registro local de conductor, vehículo, recargas, actividades, rutas e incidentes con fotografías. Usa SQLite, Xamarin.Essentials, Xam.Plugin.Media y un enfoque MVVM básico.

Propósito y alcance

Digitalizar una bitácora operativa desde el teléfono y demostrar navegación, captura de datos, persistencia y acceso a cámara. La versión actual debe tratarse como prototipo: varias funciones están incompletas y Xamarin ya no es una base vigente para una nueva publicación.

Qué resuelve

- Inicio de sesión de demostración.
- Datos de conductor y vehículo.
- CRUD parcial de conductores.
- Registro de combustible, actividades, rutas e incidentes.
- Fotografías y persistencia SQLite local.

Estado actual

El código compila conceptualmente sobre Xamarin.Forms 5 y .NET Standard 2.0, pero no se construyó en el entorno de consolidación por falta del SDK/workload compatible. Se validó la estructura XML. Hay autenticación fija de demostración, inicialización incompleta de tablas, operaciones async no esperadas, eventos duplicables, persistencia de incidentes incompleta, administración de vehículo/actividad pendiente e implementación SQLite iOS ausente.

Alcance de esta entrega

- Instalación o incorporación de la versión presente en la rama principal.
- Recorridos de uso, verificaciones y recuperación ante fallos habituales.
- Arquitectura, configuración, datos, integraciones, pruebas y despliegue.
- Límites conocidos y una ruta de evolución coherente con el estado del código.

Fuera de alcance

- Publicar la aplicación en tiendas en su estado actual.
- Usar el login de demostración o SQLite local como seguridad de producción.
- Prometer compatibilidad futura de Xamarin sin migración a .NET MAUI.

02 · GUÍA DE USUARIO

Preparación y puesta en marcha

Use únicamente datos ficticios y un dispositivo de prueba. No confíe en el prototipo como bitácora única: hasta cerrar las brechas de persistencia y respaldo, conserve un registro alternativo.

Requisitos previos

- Visual Studio con soporte Xamarin compatible, SDK Android y emulador/dispositivo.
- Permisos de cámara, galería y almacenamiento según la plataforma.
- Paquetes NuGet restaurados.
- Datos de demostración sin información laboral real.

Instalación o incorporación

1. Abrir la solución en un entorno compatible con Xamarin.Forms 5.
2. Restaurar NuGet y seleccionar Android como destino inicial.
3. Configurar un emulador y conceder permisos solo durante la prueba.
4. Compilar en Debug.
5. Recorrer pantallas sin depender de datos reales y observar excepciones SQLite.

```
# Abrir la solución en Visual Studio con Xamarin instalado
# Restaurar paquetes NuGet
# Seleccionar el proyecto Android como inicio
# Compilar y ejecutar en un emulador de prueba
```

Primera ejecución

1. Iniciar con las credenciales de demostración documentadas en el código.
2. Abrir datos del conductor y vehículo.
3. Crear un conductor ficticio y reiniciar la aplicación para probar persistencia.
4. Registrar una recarga y una actividad ficticias.
5. Crear un incidente con foto de prueba y verificar qué campos persisten.

Señal de que todo quedó bien: la navegación básica funciona en Android y los datos que la versión declara persistentes reaparecen tras reiniciar; cualquier pérdida se registra como limitación, no se oculta.

Operación paso a paso

Los recorridos siguientes están escritos para una jornada real de trabajo. Antes de confirmar una acción masiva, conviene validar primero un caso de prueba y revisar el resultado.

Ingresar al prototipo

1. Abrir la aplicación en un emulador.
2. Usar el usuario de demostración definido por la versión.
3. Confirmar que aparece la pantalla principal.
4. No reutilizar esa credencial fuera de pruebas.

Resultado esperado: se accede al entorno ficticio sin afirmar autenticación segura.

Administrar un conductor

1. Abrir la lista de conductores.
2. Crear un registro ficticio.
3. Editar un campo y guardar.
4. Eliminar solo el registro de prueba.
5. Reiniciar y comprobar qué operaciones persistieron.

Resultado esperado: el CRUD implementado se valida y cualquier fallo async queda reproducible.

Registrar una recarga

1. Abrir Recargas.
2. Completar fecha, valor/cantidad y vehículo ficticio.
3. Guardar una sola vez.
4. Volver a la lista.
5. Reiniciar para comprobar SQLite.

Resultado esperado: la recarga aparece una vez y conserva datos válidos.

Registrar actividad o ruta

1. Crear una actividad con origen, destino y fecha ficticios.
2. Guardar y revisar la lista.
3. Probar navegación hacia el detalle.
4. No asumir edición/eliminación si la pantalla aún no la implementa.

Resultado esperado: el alta y la consulta funcionan dentro del alcance actual.

Registrar un incidente con fotografía

1. Conceder permiso de cámara en el dispositivo de prueba.
2. Crear un incidente ficticio.
3. Tomar o elegir una imagen sin personas ni placas reales.
4. Guardar y reiniciar.
5. Documentar si texto, imagen o relación no persisten.

Resultado esperado: se conoce con precisión qué parte del incidente funciona y qué queda pendiente.

Lista de control operativa

- Trabajar primero con datos de prueba cuando se cambie la configuración.
- Confirmar el resultado antes de ejecutar acciones masivas o irreversibles.
- Registrar fecha, entrada, salida y mensaje exacto cuando ocurra un error.
- Cerrar sesiones y proteger credenciales al terminar la jornada.

Problemas frecuentes y recuperación

Si algo falla, conserve el mensaje exacto, la hora y la acción que lo produjo. Esa información suele ser más útil que reinstalar o borrar la configuración sin diagnóstico.

Síntoma	Causa probable	Qué revisar
No compila	Xamarin/workload o SDK ausente	Usar Visual Studio compatible o iniciar migración MAUI.
La base falla al abrir	Tablas no inicializadas o ruta DependencyService	Revisar creación await y dependencia por plataforma.
Guarda dos veces	Eventos Clicked suscritos repetidamente	Suscribir una vez y desuscribir al salir.
Dato no aparece al reiniciar	Operación async no esperada o tabla faltante	Esperar tareas, capturar errores y verificar esquema.
Foto no se guarda	Permiso, ruta o persistencia incompleta	Probar en Android y guardar una referencia local controlada.
iOS no inicia SQLite	Implementación de plataforma pendiente	No anunciar soporte iOS hasta implementarla y probarla.

Uso responsable, privacidad y seguridad

- Nunca confirmar contraseñas, tokens, claves API, perfiles del navegador ni archivos de configuración locales.
- Usar cuentas y permisos con el alcance mínimo necesario; retirar accesos cuando ya no se utilicen.
- Anonimizar correos, nombres, identificadores, órdenes y capturas antes de publicar evidencias en el portafolio.
- Revisar el historial completo de Git, no solo la última versión, antes de hacer público el repositorio.
- Sustituir el login fijo por autenticación real antes de manejar datos personales.
- Cifrar o proteger la base, minimizar fotos y definir consentimiento/retención para incidentes.

Preguntas habituales

¿Está lista para producción?

No. Es un prototipo con brechas explícitas de persistencia, seguridad, pruebas y compatibilidad.

¿Funciona en iOS?

La implementación SQLite de iOS está pendiente; no debe afirmarse soporte hasta construir y probar.

¿Por qué migrar a MAUI?

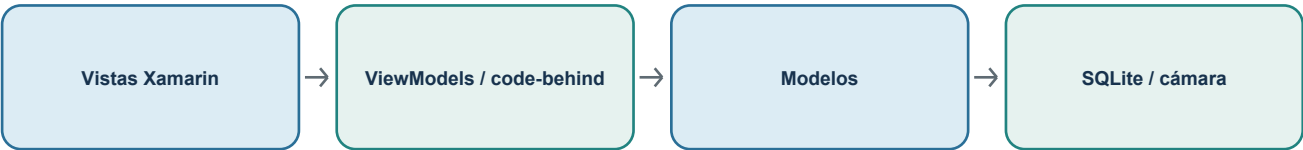
Xamarin dejó de ser la ruta soportada para nuevas aplicaciones; .NET MAUI permite actualizar plataforma, dependencias y ciclo de publicación.

03 · REFERENCIA TÉCNICA

Arquitectura y componentes

La solución usa modelos de dominio simples, vistas Xamarin.Forms, SQLite asíncrono y DependencyService para acceder a funciones por plataforma. El MVVM es parcial: parte de la lógica todavía vive en code-behind.

Vista de arquitectura



La persistencia y los servicios de plataforma deben encapsularse y esperarse de forma asíncrona. Hoy varias rutas mezclan UI y acceso a datos.

Responsabilidad de los componentes

Componente	Responsabilidad	Tecnología o archivo
Modelos	Conductor, vehículo, recarga, actividad e incidente	Empleado, T_Conductor, T_Vehiculo, T_Recarga, T_Actividad, T_Incidente
Vistas	Pantallas y navegación	XAML y code-behind
ViewModels	Estado de presentación parcial	FotoViewModel y otros
Persistencia	Operaciones SQLite asíncronas	SQLite-net / servicios
Plataforma	Ruta DB, cámara, galería	DependencyService, Essentials, Media
Proyectos nativos	Android e iOS	Implementaciones por plataforma

Configuración, datos e integraciones

Parámetros importantes

Parámetro / recurso	Uso	Tratamiento recomendado
Login demo	Acceso fijo de prototipo	Eliminar antes de producción.
Ruta SQLite	Base local por plataforma	Implementar y probar Android/iOS.
Permisos cámara/galería	Captura de incidente	Solicitar en contexto y explicar propósito.
Imágenes externas	Recursos de interfaz	Empaquetar o usar origen controlado.
Identificador/firma	Paquete móvil	Custodiar claves fuera de Git.

Persistencia y archivos generados

- SQLite guarda datos locales; la creación de todas las tablas debe completarse antes de confiar en ella.
- Las fotografías pueden contener datos personales y requieren ruta, retención y eliminación coherentes.
- No existe una estrategia completa de sincronización o respaldo.
- Las operaciones async deben esperarse y reportar errores al usuario.

Integraciones externas

Integración	Función	Riesgo operativo
SQLite-net	Persistencia local	Esquema, concurrencia y tareas no esperadas.
Xamarin.Essentials	Funciones del dispositivo	Permisos y compatibilidad.
Xam.Plugin.Media	Cámara y galería	Mantenimiento del paquete y privacidad.
Android/iOS	Ejecución móvil	SDK, firma y diferencias de plataforma.
Recursos externos	Imágenes u otros activos	Disponibilidad/licencia.

Flujos internos y decisiones de diseño

Persistencia de un formulario

1. Validar campos en la vista.
2. Construir o actualizar el modelo.
3. Esperar la operación SQLite.
4. Capturar y mostrar un error útil.
5. Recargar desde la base y navegar solo si fue exitoso.

Navegar antes de await puede hacer creer que se guardó un dato que realmente falló.

Captura de incidente

1. Solicitar permiso en contexto.
2. Capturar o elegir foto.
3. Copiarla a almacenamiento administrado.
4. Guardar metadatos y relación con el incidente.
5. Comprobar lectura y eliminación.

Una ruta temporal de cámara puede desaparecer; la aplicación debe hacerse responsable de la copia que conserva.

Criterios de diseño

- Separar configuración y credenciales del código que se versiona.
- Mantener una entrada clara para operar y otra para diagnóstico o pruebas.
- Preferir fallos explícitos a continuar con datos parciales o inventados.
- Conservar trazabilidad suficiente para reconstruir qué ocurrió sin exponer información sensible.

Desarrollo, pruebas y mantenimiento

Entorno de desarrollo

- Visual Studio compatible con Xamarin.Forms y las cargas de trabajo móviles requeridas.
- SDK de Android, emulador o dispositivo de prueba y herramientas de depuración.
- Git y configuración de firma fuera del repositorio.

```
# Restaurar NuGet desde Visual Studio
# Compilar Debug para Android
# Ejecutar análisis estático y pruebas una vez migrado a SDK moderno
```

Estrategia de pruebas

- Actualmente no hay suite automatizada: es una brecha crítica.
- Añadir pruebas de validación y repositorios antes de nuevas funciones.
- Pruebas de integración SQLite para crear, actualizar, eliminar y reiniciar.
- Pruebas de UI en Android; habilitar iOS solo con implementación y dispositivo/simulador.

Registro, diagnóstico y soporte

- Conservar el mensaje completo y la hora del evento; evitar capturas que muestren credenciales o datos personales.
- Distinguir un error local de un fallo del servicio externo comprobando conectividad, permisos y cuotas.
- Repetir el caso con una entrada pequeña y controlada antes de cambiar dependencias o borrar datos.

Mantenimiento recomendado

- Revisar dependencias y servicios externos de manera periódica, probando la actualización en una rama separada.
- Actualizar este manual cuando cambien comandos, configuración, estructura de datos o flujo de usuario.
- Mantener ejemplos sin secretos y una prueba mínima que permita validar la instalación.
- Etiquetar versiones estables y documentar las alternativas como ramas, no como reemplazos ambiguos.

Despliegue y continuidad

Ruta de despliegue

1. Abrir la solución y restaurar paquetes en un entorno compatible.
2. Configurar identificador, permisos y firma para una compilación de prueba.
3. Instalar en un dispositivo controlado y recorrer las pantallas principales.
4. Verificar persistencia local, cámara y navegación.
5. Generar un paquete de distribución solo después de corregir los límites críticos señalados.

Respaldo y recuperación

- Respaldar solo datos necesarios y cifrar cualquier copia que contenga información personal u operativa.
- Probar la restauración en una ubicación distinta antes de confiar en el respaldo.
- Conservar una copia de la configuración sin secretos y registrar por separado dónde se administran las credenciales.

Criterios de aceptación

- Una instalación limpia puede completar el recorrido principal descrito en este manual.
- Los errores de credenciales, permisos o red se informan sin filtrar secretos.
- Los archivos temporales y los datos generados permanecen fuera del repositorio.
- La persona responsable puede identificar versión, configuración y evidencia de la última ejecución correcta.

Límites conocidos y evolución

Límites de la versión documentada

- Xamarin ya no es una plataforma vigente para nuevas entregas.
- Autenticación fija de demostración.
- Persistencia y manejo asíncrono incompletos.
- Incidentes, vehículo, actividad e iOS tienen funciones pendientes.
- Sin pruebas automatizadas ni compilación validada en el entorno actual.

Hoja de ruta sugerida

Prioridad	Mejora	Resultado esperado
Crítica	Migrar a .NET MAUI y SDK soportado	Recuperar compilación, seguridad y ciclo de publicación.
Crítica	Completar esquema SQLite y esperar todas las operaciones	Evitar pérdida silenciosa de datos.
Alta	Autenticación y protección de datos	Eliminar credencial fija y reducir exposición.
Alta	Terminar incidentes, vehículo e iOS o reducir alcance	Alinear promesa con funciones reales.
Media	Suite de pruebas y repositorios MVVM	Separar UI y facilitar mantenimiento.

Versiones y ramas

Rama / versión	Papel en el historial	Recomendación
master · Xamarin.Forms 5	Prototipo histórico documentado	Conservar como línea de origen.
Futura migración MAUI	Reimplementación recomendada	Crear en rama nueva con paridad funcional verificada.

Entrega a otra persona

Este apartado resume lo mínimo que debe conocer quien reciba el mantenimiento. La intención es que el proyecto pueda retomarse sin depender de la memoria del autor.

Paquete de traspaso

- Repositorio, rama y etiqueta exacta que se consideran estables.
- Lista de servicios externos, responsables de las cuentas y procedimiento para rotar accesos.
- Configuración de ejemplo sin secretos y ubicación de los datos persistentes.
- Resultado de la última prueba, limitaciones abiertas y próximos cambios recomendados.
- Un caso de prueba anonimizado que permita recorrer el flujo de principio a fin.

Glosario breve

Término	Significado en este proyecto
Xamarin.Forms	Framework móvil usado por el prototipo.
.NET MAUI	Plataforma moderna recomendada para la migración.
MVVM	Separación entre vista, estado de presentación y modelo.
DependencyService	Mecanismo Xamarin para código específico de plataforma.
SQLite	Base de datos local embebida.

Cierre

La mejor señal de madurez no es que el proyecto nunca falle, sino que otra persona pueda instalarlo, entender sus límites y recuperar una operación sin improvisar.

Documento mantenido junto al código fuente. Las decisiones operativas que cambien el comportamiento deben reflejarse tanto en el código como en esta guía.